

1 Password Hashing & Bcrypt

Database ma kahile pani raw password save garnu hudaina. Yadi database leak vayo vane sabai user ko account danger ma parcha. Hami **Bcrypt** algorithm use garchham, jasle plain-text password lai cryptographic *hash* ma badalchha ra uniquely solid *salt* layer thapchha.

2 Registration flow ma Hash garne ra Login flow ma Compare garne method:

JavaScript

```
const bcrypt = require('bcrypt');

const hashPassword = async (plainPassword) => {
  const saltRounds = 10; // Computational cost buffer
  const hashed = await bcrypt.hash(plainPassword, saltRounds);
  return hashed;
};

const comparePassword = async (plainPassword, hashedPassword) => {
  const isMatch = await bcrypt.compare(plainPassword, hashedPassword);
  return isMatch; // Returns true or false
};
```

3 2. JWT (JSON Web Token) Architecture

JWT le stateless authentication algorithms handle garchha. Server le password valid vaysi sign garera user lai dinchha, ra user le harek request ma payload token bokera aauchha.

JWT ma thin separate structures hunchhan: `Header.Payload.Signature`

JavaScript

```
const jwt = require('jsonwebtoken');

// Generate Token
const generateAccessToken = (userId) => {
  return jwt.sign({ id: userId }, process.env.JWT_SECRET, { expiresIn: '15m' });
};
```

4 3. Register & Login Implementation (The Baseline)

Aaba user collection abstraction code modular style ma build garum:

JavaScript

```
const express = require('express');
const router = express.Router();
const bcrypt = require('bcrypt');
const jwt = require('jsonwebtoken');
```

```
// Mock User DB Array (Tapaila mongoose/prisma use garda DB query rakhne)
const users = [];

// 1. REGISTER
router.post('/register', async (req, res) => {
  try {
    const { email, password } = req.body;

    // Check user existence
    const userExists = users.find(u => u.email === email);
    if (userExists) return res.status(400).json({ error: "Email target
vayo, user exist cha!" });

    // Password hash garne
    const hashedPassword = await bcrypt.hash(password, 10);

    const newUser = { id: Date.now().toString(), email, password:
hashedPassword };
    users.push(newUser);

    res.status(201).json({ message: "Registration Complete!" });
  } catch (err) {
    res.status(500).json({ error: "Server down!" });
  }
});

// 2. LOGIN
router.post('/login', async (req, res) => {
  const { email, password } = req.body;
  const user = users.find(u => u.email === email);
  if (!user) return res.status(400).json({ error: "Invalid Credentials!"
});

  // Validate structure match
  const isPasswordValid = await bcrypt.compare(password, user.password);
  if (!isPasswordValid) return res.status(400).json({ error: "Invalid
Credentials!" });

  // Build unique authorization assets
  const accessToken = generateAccessToken(user.id);
  const refreshToken = jwt.sign({ id: user.id },
process.env.REFRESH_SECRET, { expiresIn: '7d' });

  // Refresh token database va secure HTTP-only cookie ma halne
  user.refreshToken = refreshToken;

  res.json({ accessToken, refreshToken });
});
```

5 4. Protected Routes Middleware

Yadi kunai endpoint login vako user le matra chalauna paune banaunu chha vane hami **Auth Middleware** runtime runtime inject garchham:

JavaScript

```

const authenticateToken = (req, res, next) => {
  // Header format: "Bearer TOKEN_STRING"
  const authHeader = req.headers['authorization'];
  const token = authHeader && authHeader.split(' ')[1];

  if (!token) return res.status(401).json({ error: "Access Denied. Token chaina!" });

  jwt.verify(token, process.env.JWT_SECRET, (err, decodedUser) => {
    if (err) return res.status(403).json({ error: "Invalid or Expired Token!" });

    req.user = decodedUser; // Inject decoded user id to request context
    next();
  });
};

// Protected endpoint consumption sample
router.get('/dashboard', authenticateToken, (req, res) => {
  res.json({ message: `Welcome User ID: ${req.user.id}. Access granted to secure resources.` });
});

```

6 5. Refresh Tokens: Multi-Session Architecture

Access token thora samaya (15m) ma expire hunchha. Yo security strict rakhna ho. Tarah user le harek 15 minutes ma login gari rakhnu naparos vanera **Refresh Token** use hunchha.

JavaScript

```

router.post('/refresh', (req, res) => {
  const { token } = req.body; // Ideally dynamic secure HTTP-only cookies
  bata line
  if (!token) return res.status(401).json({ error: "Refresh token pathau!" });

  const user = users.find(u => u.refreshToken === token);
  if (!user) return res.status(403).json({ error: "Token active chaina database ma!" });

  jwt.verify(token, process.env.REFRESH_SECRET, (err, decoded) => {
    if (err) return res.status(403).json({ error: "Token fail or altered!" });

    // Nayan Access token generate garera pathaune
    const newAccessToken = generateAccessToken(decoded.id);
    res.json({ accessToken: newAccessToken });
  });
});

```

7 6. Email Verification, Forgot & Reset Password Lifecycle

Yo lifecycle system scale up garda dherai security features add garnu parne component ho:

[Register] —> Generate Verify Token —> Mail via Nodemailer —> Click Link —> Mark as Active Verified

[Forgot Pwd] —> Generate Safe String Hash + Expiry (e.g., 1 hour) —> Send Mail —> Reset via Target Token

8 Complete Code Structure Strategy:

JavaScript

```
const crypto = require('crypto'); // Built-in utility object string hashes generation ko lagi
```

```
// 1. FORGOT PASSWORD (Trigger Request)
router.post('/forgot-password', async (req, res) => {
  const { email } = req.body;
  const user = users.find(u => u.email === email);
  if (!user) return res.status(404).json({ error: "User found vayana!" });

  // Plain crypto string dynamically scaling tokens
  const resetToken = crypto.randomBytes(32).toString('hex');

  // Hash text to prevent database manipulation leaks
  user.passwordResetToken =
    crypto.createHash('sha256').update(resetToken).digest('hex');
  user.passwordResetExpires = Date.now() + 3600000; // 1 hour validation active window

  const resetURL =
    `${req.protocol}://${req.get('host')}/api/v1/users/reset-password/${resetToken}`;

  // Nodemailer flow injection strategy here:
  // sendEmail({ email: user.email, subject: "Password Reset Link",
  message: resetURL });

  res.json({ message: "Reset link ready! Mail sent successfully (Simulation).", link: resetURL });
});

// 2. RESET PASSWORD (Verification and Action Update)
router.post('/reset-password/:token', async (req, res) => {
  // URL bata token liyera algorithm dynamically compare garne
  const hashedToken =
    crypto.createHash('sha256').update(req.params.token).digest('hex');

  const user = users.find(u => u.passwordResetToken === hashedToken &&
    u.passwordResetExpires > Date.now());
  if (!user) return res.status(400).json({ error: "Token validation failed or expired!" });

  // Nayan password clean replacement setup
  user.password = await bcrypt.hash(req.body.password, 10);

  // Clear the reset validation parameters
```

```
user.passwordResetToken = undefined;
user.passwordResetExpires = undefined;

res.json({ message: "Password updated successfully! Aaba login garna
saknu hunchha." });
});
```